

Connect Datadog Monitoring JMX auto discovery and Datadog Datastreams setup

Installation

Connect setup

In the connect manifest under pod template, add the following:

```
1  podTemplate:
2
3    annotations:
4
5      ad.datadoghq.com/connect-ibm-mq-appeng.check_names: '["confluent_platform"]'
6
7      ad.datadoghq.com/connect-ibm-mq-appeng.init_configs: |
8
9        [
10
11          {
12
13            "is_jmx": true,
14
15            "collect_default_metrics": true,
16
17            "service_check_prefix": "confluent",
18
19            "new_gc_metrics": true,
20
21            "collect_default_jvm_metrics": true
22
23          }
24
25        ]
26
27      ad.datadoghq.com/connect-ibm-mq-appeng.instances: |
28
29        [
30
31          {
32
33            "host": "%host%",
34
35            "port": 7203
36
37          }
38
39        ]
40
41
```

NOTE: In this example connect-ibm-mq-appeng maps to the name of the connect cluster, yours may be different so update accordingly.

Deploy the updated connect manifest file.

Datadog setup

- Create namespace datadog

- Create a secret (You will need to create an API key in datadog **NOTE NOT APPLICATION KEY**)

```
kubectl create secret generic datadog-secret --from-literal=api-key='datadogkeyhere' -n datadog
```

- Create a values.yaml file with the following:

```
1 datadog:
2
3   site: datadoghq.eu
4
5   apiKeyExistingSecret: datadog-secret
6
7   apm:
8
9     enabled: true
10
11   processAgent:
12
13     enabled: true
14
15   logs:
16
17     enabled: true
18
19   kubeStateMetricsEnabled: true
20
21   clusterChecksEnabled: true
22
23   prometheusScrape:
24
25     enabled: true
26
27   kubelet:
28
29     host:
30
31     valueFrom:
32
33       fieldRef:
34
35         fieldPath: status.hostIP
36
37     tlsVerify: false
38
39   containers:
40
41     logs: true
42
43   tags:
44
45     - "env:prod"
46
47   clusterAgent:
48
49     enabled: true
50
51   apiKeyExistingSecret: datadog-secret
```

- Run helm install datadog -f values.yaml --set datadog.apiKeyExistingSecret=datadog-secret datadog/datadog

Ensure pods are running `kubectl get pods --namespace data-dog`

- Create file called `datadog-rbac.yaml` with contents below:

```
1  apiVersion: v1
2
3  kind: ServiceAccount
4
5  metadata:
6
7    name: datadog-agent
8
9    namespace: datadog
10
11  ---
12
13  apiVersion: rbac.authorization.k8s.io/v1
14
15  kind: ClusterRole
16
17  metadata:
18
19    name: datadog-agent-role
20
21  rules:
22
23    - apiGroups: [""]
24
25      resources:
26
27        - nodes
28
29        - nodes/proxy
30
31        - nodes/stats
32
33        - nodes/metrics
34
35        - services
36
37        - endpoints
38
39        - pods
40
41      verbs: ["get", "list", "watch"]
42
43    - nonResourceURLs: ["/metrics"]
44
45      verbs: ["get"]
46
47  ---
48
49  apiVersion: rbac.authorization.k8s.io/v1
50
51  kind: ClusterRoleBinding
52
53  metadata:
54
55    name: datadog-agent-role-binding
56
57  roleRef:
58
59    apiGroup: rbac.authorization.k8s.io
60
61    kind: ClusterRole
62
```

```
63   name: datadog-agent-role
64
65   subjects:
66
67   - kind: ServiceAccount
68
69     name: datadog-agent
70
71     namespace: datadog
```

- Run apply command
- Create token openssl rand -hex 16
- Create secret for the token

```
kubectl create secret generic datadog-cluster-agent-token -n datadog \
    --from-literal=token=b7ac4c2fd99350ea89c40cac3a5d437b
```

- Create a datadog-agent.yaml file with the following:

```
1  apiVersion: apps/v1
2
3  kind: DaemonSet
4
5  metadata:
6
7    name: datadog-agent
8
9    namespace: datadog
10
11  spec:
12
13    selector:
14
15      matchLabels:
16
17        app: datadog-agent
18
19    template:
20
21      metadata:
22
23        labels:
24
25          app: datadog-agent
26
27      spec:
28
29        serviceAccountName: datadog-agent
30
31      containers:
32
33        - name: agent
34
35          image: "gcr.io/datadoghq/agent:latest-jmx"
36
37          env:
38
39            - name: DD_API_KEY
40
41              valueFrom:
42
43                secretKeyRef:
```

```
45         name: datadog-secret
46
47         key: api-key
48
49     - name: DD_CLUSTER_AGENT_USE_TLS
50
51       value: "true"
52
53     - name: DD_CLUSTER_AGENT_URL
54
55       value: "https://datadog-cluster-agent.datadog.svc:5005"
56
57     - name: DD_CLUSTER_AGENT_AUTH_TOKEN
58
59       valueFrom:
60
61         secretKeyRef:
62
63           name: datadog-cluster-agent-token
64
65           key: token
66
67     - name: DD_SITE
68
69       value: "datadoghq.eu"
70
71     - name: DD_LOGS_ENABLED
72
73       value: "false"
74
75     - name: DD_APM_ENABLED
76
77       value: "false"
78
79     - name: DD_DOGSTATSD_NON_LOCAL_TRAFFIC
80
81       value: "true"
82
83     - name: DD_COLLECT_KUBERNETES_EVENTS
84
85       value: "true"
86
87     - name: DD_PROMETHEUS_SCRAPE_ENABLED
88
89       value: "true"
90
91     - name: DD_KUBELET_TLS_VERIFY
92
93       value: "false"
94
95     - name: DD_CLUSTER_AGENT_ENABLED
96
97       value: "true"
98
99     - name: DD_ORCHESTRATOR_EXPLORER_ENABLED
100
101       value: "false"
102
103     - name: DD_KUBERNETES_POD_TAGGER_ENABLED
104
105       value: "true"
106
107     - name: "DD_REMOTE_CONFIGURATION_ENABLED"
108
109       value: "false"
110
111     - name: DD_ORCHESTRATOR_CONFIG_ORCHESTRATOR_URL
```

```
112         value: "https://orchestrator.datadoghq.eu"
113
114     - name: DD_HOSTNAME
115
116       valueFrom:
117
118         fieldRef:
119
120           fieldPath: spec.nodeName
121
122     - name: DD_KUBERNETES_KUBELET_HOST
123
124       valueFrom:
125
126         fieldRef:
127
128           fieldPath: status.hostIP
129
130   ports:
131
132     - containerPort: 8125
133
134       name: dogstatsd
135
136       protocol: UDP
137
138     - containerPort: 8126
139
140       name: traceport
141
142       protocol: TCP
143
144     - containerPort: 5005
145
146       name: agentapi
147
148       protocol: TCP
149
150   volumeMounts:
151
152     - name: dockersocket
153
154       mountPath: /var/run/docker.sock
155
156     - name: procdir
157
158       mountPath: /host/proc
159
160       readOnly: true
161
162     - name: cgroups
163
164       mountPath: /host/sys/fs/cgroup
165
166       readOnly: true
167
168   volumes:
169
170     - name: dockersocket
171
172       hostPath:
173
174         path: /var/run/docker.sock
175
176     - name: procdir
```

```
179         hostPath:
180
181         path: /proc
182
183     - name: cgroups
184
185         hostPath:
186
187         path: /sys/fs/cgroup
```

- Run apply command
- Create datadog-cluster-agent.yaml file with contents, for DD_CLUSTER_NAME replace the value with your cluster name:

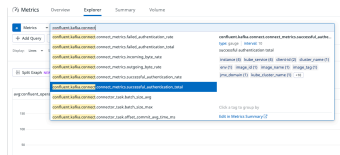
```
1  apiVersion: apps/v1
2
3  kind: Deployment
4
5  metadata:
6
7      name: datadog-cluster-agent
8
9      namespace: datadog
10
11      labels:
12
13          app: datadog-cluster-agent
14
15  spec:
16
17      replicas: 1
18
19      selector:
20
21          matchLabels:
22
23              app: datadog-cluster-agent
24
25  template:
26
27      metadata:
28
29          labels:
30
31              app: datadog-cluster-agent
32
33      spec:
34
35          serviceAccountName: datadog-cluster-agent
36
37          containers:
38
39              - name: cluster-agent
40
41                  image: gcr.io/datadoghq/cluster-agent:latest
42
43                  env:
44
45                      - name: DD_API_KEY
46
47                          valueFrom:
48
49                              secretKeyRef:
50
```

```

51         name: datadog-secret
52
53         key: api-key
54
55     - name: DD_CLUSTER_NAME
56
57       value: "mayat-cfk-test"
58
59     - name: DD_CLUSTER_AGENT_AUTH_TOKEN
60
61       valueFrom:
62
63         secretKeyRef:
64
65           name: datadog-cluster-agent-token
66
67           key: token
68
69     - name: DD_SITE
70
71       value: "datadoghq.eu"
72
73     - name: DD_ORCHESTRATOR_EXPLORER_ENABLED
74
75       value: "true"
76
77     - name: DD_ORCHESTRATOR_CONFIG_ORCHESTRATOR_URL
78
79       value: "https://orchestrator.datadoghq.eu"

```

- Apply then check all pods are running in datadog namespace
- Check agents status in datadog at url <https://app.datadoghq.eu/fleet>
- Check metrics are appearing <https://app.datadoghq.eu/metric/explorer>



Datadog datastreams setup

NB: The setup steps were taken from [Tracing Java Applications](#) and applied to the Connect CFK setup.

If you want to set up DataDog datastreams monitoring after the JMX setup you can make the following changes:

- Download the dd-java-agent.jar from <https://dtdg.co/latest-java-tracer>, then update the Connect docker image

```

1 RUN mkdir -p /usr/share/java/dd-java-agent
2

```



```
3 COPY dd-java-agent.jar /usr/share/java/dd-java-agent/dd-java-agent.jar
```

- Update the datadog-agent yaml file and set

```
1 - name: DD_APM_ENABLED
2
3   value: "true"
```

Apply the change

- Next expose 8126 apm port by creating a service and applying it

```
1 apiVersion: v1
2
3 kind: Service
4
5 metadata:
6
7   name: datadog-agent
8
9   namespace: datadog
10
11 spec:
12
13   selector:
14
15     app: datadog-agent
16
17   ports:
18
19     - name: apm
20
21       port: 8126
22
23       targetPort: 8126
```

- Update connect yaml with jvm overrides and apply

```
1 jvm:
2
3   - "-javaagent:/usr/share/java/dd-java-agent/dd-java-agent.jar"
4
5   - "-Ddd.profiling.enabled=true"
6
7   - "-Ddd.logs.injection=true"
8
9   - "-Ddd.service=connect-ibm-mq-appeng"
10
11   - "-Ddd.env=dev"
12
13   - "-Ddd.version=1.0"
14
15   - "-Ddd.agent.host=datadog-agent.datadog.svc.cluster.local"
```

NB: your agent host URL might be different

References

-  [Autodiscovery with JMX](#)
-  [Confluent Platform](#)
-  [Monitor Confluent Platform with Datadog | Datadog](#)
- Confluent platform integration tile - this can be installed, cloned and configured
- <https://app.datadoghq.eu/marketplace/integration/confluent-platform?search=confluent%20platform>
-  [Track and improve the performance of streaming data pipelines with Datadog Data Streams Monitoring | Datadog](#)
-  [Confluent Platform](#)
-  [Tracing Java Applications](#)

Only confluent.kafka.connect will be collected, as we are only running connect